

Database Design for a Profile Service

Normalisation, ERD, SQL, views, stored procedures and a logging trigger

Overview

This project is based on a database module I took at university. The task was to design and build the database behind a profile service for a community app. I started with the raw profile data, normalised it to Third Normal Form, drew the entity relationship diagram, then built the tables, a view, CRUD stored procedures, and a trigger in Microsoft SQL Server.

Disclaimer

For this portfolio version I changed the app name, the entity names, the sample data and the server details. Margins is a fictional app. The database design, the normalisation steps and the SQL are the ones I wrote for my coursework.

Scenario

Margins is a fictional app for people who love reading. The profile service stores each user's profile, their favourite genres, and the city they are based in. A user can pick many genres, and each genre can be picked by many users. The service needs tables to store the data, stored procedures to manage it, a view to show a full profile, and a trigger to log when a new user joins.

Normalisation

Normalisation splits data into tables so there is no repetition and the data stays consistent. I worked from the raw profile data up to Third Normal Form (3NF), one step at a time.

Unnormalised Form (UNF)

The raw data has a repeating group. A user can have many favourite genres, shown here in brackets.

USERS	UserID FirstName LastName Username Email Bio City (FavouriteGenres)
--------------	--

First Normal Form (1NF)

The repeating group is moved into its own table. Each row now holds one value, with no repeating group.

USERS	UserID FirstName LastName Username Email Bio City
--------------	---

USERGENRE	UserID FavouriteGenre
------------------	--------------------------

Second Normal Form (2NF)

The genre name only depends on the genre, not on the user. I moved it into a GENRE table to remove the partial dependency.

USERS	UserID FirstName LastName Username Email Bio City
USERGENRE	UserID GenreID
GENRE	GenreID GenreName

Third Normal Form (3NF)

The city name depends on the city, not the user. A CITY table removes this transitive dependency, so a city is stored once and reused.

USERS	UserID FirstName LastName Username Email Bio *CityID
CITY	CityID CityName
GENRE	GenreID GenreName
USERGENRE	*UserID *GenreID

Final Entity Relationship Diagram

The final ERD brings the normalised tables together. USERGENRE is the linking table that resolves the many to many relationship between users and genres. USERLOG is added later for the logging trigger.

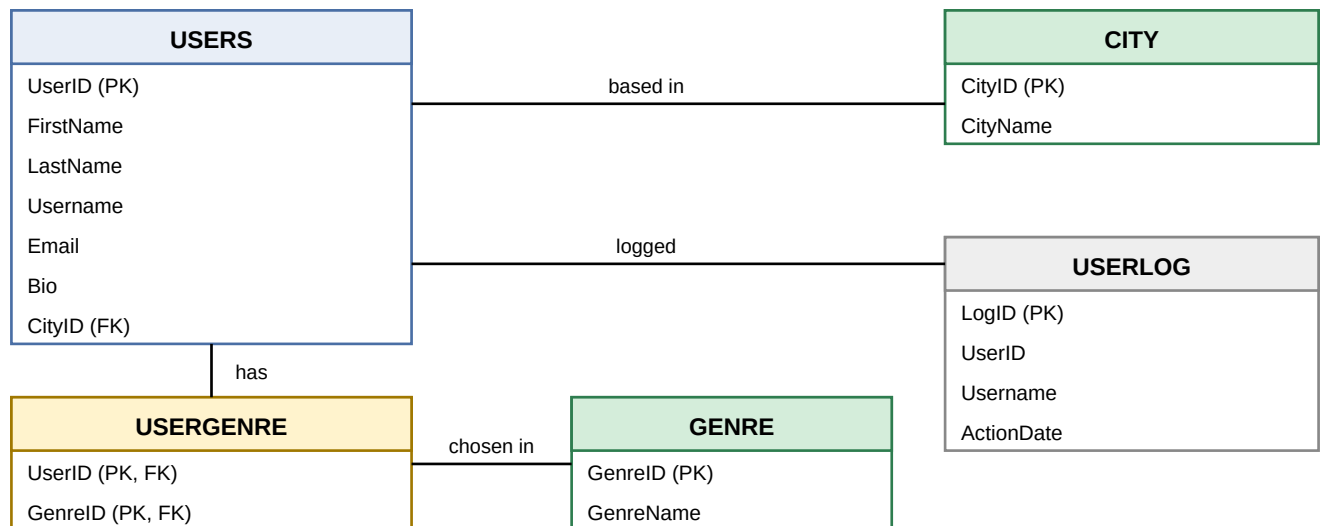


Figure 1. Entity relationship diagram for the Margins profile database

- **USERS** to **CITY**: many users can be based in one city.
- **USERS** to **USERGENRE** to **GENRE**: a user can have many genres, and a genre can be chosen by many users.
- **USERS** to **USERLOG**: each new user creates one log record.

Credential Handling

Design note. This profile database does not store passwords. In the full system, authentication is handled by a separate service, so the profile data holds no credentials. If a credential ever had to be stored here, it would be kept as a secure hash, never as plaintext. Keeping login details out of the profile store lowers the risk if the profile data is ever exposed.

Data Dictionary

The field definition grid below shows the design of each table before building it.

USERS

Attribute	Data Type	Key	Optional	Description
UserID	INT	Primary Key	No	Unique ID for each user
FirstName	VARCHAR(50)		No	User first name
LastName	VARCHAR(50)		No	User last name
Username	VARCHAR(50)		No	Unique display name
Email	VARCHAR(100)		No	User email address
Bio	VARCHAR(500)		Yes	Short profile description
CityID	INT	Foreign Key	Yes	Links to the CITY table

CITY, GENRE and USERGENRE

Table	Attribute	Data Type	Key
CITY	CityID	INT	Primary Key
CITY	CityName	VARCHAR(100)	No
GENRE	GenreID	INT	Primary Key
GENRE	GenreName	VARCHAR(100)	No
USERGENRE	UserID	INT	Primary Key, Foreign Key
USERGENRE	GenreID	INT	Primary Key, Foreign Key

Creating the Tables

I built the tables in Microsoft SQL Server under their own schema. The linking table uses a composite primary key made of both foreign keys, so a user cannot pick the same genre twice.

```

CREATE TABLE margins.USERS (
    UserID    INT IDENTITY(1,1) PRIMARY KEY,
    FirstName VARCHAR(50)  NOT NULL,
    LastName  VARCHAR(50)  NOT NULL,
    Username  VARCHAR(50)  UNIQUE NOT NULL,
    Email     VARCHAR(100) UNIQUE NOT NULL,
    Bio       VARCHAR(500) NULL,
    CityID    INT NULL
);

CREATE TABLE margins.CITY (
    CityID    INT IDENTITY(1,1) PRIMARY KEY,
    CityName  VARCHAR(100) NOT NULL
);

CREATE TABLE margins.GENRE (
    GenreID   INT IDENTITY(1,1) PRIMARY KEY,
    GenreName VARCHAR(100) NOT NULL
);

CREATE TABLE margins.USERGENRE (
    UserID    INT NOT NULL,
    GenreID   INT NOT NULL,
    PRIMARY KEY (UserID, GenreID),
    FOREIGN KEY (UserID) REFERENCES margins.USERS(UserID),
    FOREIGN KEY (GenreID) REFERENCES margins.GENRE(GenreID)
);

ALTER TABLE margins.USERS
ADD FOREIGN KEY (CityID) REFERENCES margins.CITY(CityID);

```

Adding Demo Data

I added sample rows so the view and the procedures have data to work with.

```

INSERT INTO margins.CITY (CityName) VALUES
('Bristol'), ('Bath'), ('Cardiff');

INSERT INTO margins.GENRE (GenreName) VALUES
('Fiction'), ('History'), ('Science');

INSERT INTO margins.USERS (FirstName, LastName, Username, Email, Bio, CityID) VALUES
('Maya', 'Okonkwo', 'maya', 'maya@margins.example', 'Loves long novels', 1),
('Leo', 'Marsh', 'leo', 'leo@margins.example', 'Reads popular science', 2),
('Priya', 'Shah', 'priya', 'priya@margins.example', 'Short story fan', 1);

INSERT INTO margins.USERGENRE (UserID, GenreID) VALUES
(1, 1), (1, 2), -- Maya: Fiction and History
(2, 3), -- Leo: Science
(3, 1); -- Priya: Fiction

```

Building the View

A view joins the tables so the full profile can be read in one query. I used explicit JOINS so the links between tables are clear and easy to follow.

```
CREATE VIEW margins.UserProfileView AS
SELECT u.FirstName, u.LastName, u.Username, u.Email, u.Bio,
       c.CityName, g.GenreName
FROM margins.USERS u
JOIN margins.CITY      c ON u.CityID = c.CityID
JOIN margins.USERGENRE ug ON u.UserID = ug.UserID
JOIN margins.GENRE     g ON ug.GenreID = g.GenreID;
```

Running `SELECT * FROM margins.UserProfileView` returns one row per user and genre, with the city name joined in. Maya appears twice, once for Fiction and once for History, which is expected for a many to many link.

Stored Procedures (CRUD)

Stored procedures handle the create, read, update and delete actions on the USERS table. Using them keeps the SQL in one place and stops the app from running raw queries against the tables.

```
CREATE PROCEDURE margins.AddUser
    @FirstName VARCHAR(50), @LastName VARCHAR(50),
    @Username VARCHAR(50), @Email VARCHAR(100),
    @Bio VARCHAR(500), @CityID INT
AS
BEGIN
    INSERT INTO margins.USERS (FirstName, LastName, Username, Email, Bio, CityID)
    VALUES (@FirstName, @LastName, @Username, @Email, @Bio, @CityID);
END;

CREATE PROCEDURE margins.GetUser @UserID INT
AS
BEGIN
    SELECT * FROM margins.USERS WHERE UserID = @UserID;
END;

CREATE PROCEDURE margins.UpdateUser @UserID INT, @Bio VARCHAR(500), @CityID INT
AS
BEGIN
    UPDATE margins.USERS
    SET Bio = @Bio, CityID = @CityID
    WHERE UserID = @UserID;
END;

CREATE PROCEDURE margins.DeleteUser @UserID INT
AS
BEGIN
    DELETE FROM margins.USERS WHERE UserID = @UserID;
END;
```

I tested each procedure by adding a user, reading it back, updating the bio and city, then deleting it and checking the row was gone.

Logging Trigger

A trigger logs every new user automatically. When a row is inserted into USERS, the trigger writes a record into a separate log table with the time it happened. This gives a simple audit trail without any extra work from the app.

```

CREATE TABLE margins.UserLog (
    LogID      INT IDENTITY(1,1) PRIMARY KEY,
    UserID     INT,
    Username   VARCHAR(50),
    ActionDate DATETIME
);

CREATE TRIGGER margins.LogNewUser
ON margins.USERS
AFTER INSERT
AS
BEGIN
    INSERT INTO margins.UserLog (UserID, Username, ActionDate)
    SELECT UserID, Username, GETDATE()
    FROM inserted;
END;

```

The inserted table holds the new rows. Reading from it means the trigger logs every user added in one statement, even if several are added at once.

What I Would Improve Now

- Add indexes on Email and Username, since these are used to look users up.
- Add ON DELETE rules so that removing a user also clears their rows in USERGENRE.
- Return the new UserID from AddUser so the app knows which row it created.
- Expand the log to record updates and deletes, not just new users.

Skills Demonstrated

- Database normalisation to 3NF
- Entity relationship modelling
- SQL table design with keys and constraints
- Views, stored procedures and triggers
- Microsoft SQL Server
- Security aware data design

Outcome

This project shows how I design a database from raw data to a working build. It covers the full process: normalising the data, modelling it, then building the tables, a view, CRUD procedures and a logging trigger in SQL Server. The design keeps data consistent and keeps login details out of the profile store.